```
clear; clc; close all;
no_bus=input('Total no of bus is: ')
P = zeros(1,no_bus); Q = zeros(1,no_bus);
V = zeros(1,no_bus); delta = zeros(1,no_bus);
for i = 1:no_bus
 fprintf(sprintf('--- Bus %d ---\n', i));
 P(i) = input(sprintf('P(%d) in pu (type inf if unknown): ', i));
 Q(i) = input(sprintf('Q(%d) in pu (type inf if unknown): ', i));
 V(i) = input(sprintf('V(%d) in pu (type inf if unknown): ', i));
 delta(i) = input(sprintf('delta(%d) in rad (type inf if unknown): ', i));
 end
V(isinf(V)) = 1;
delta(isinf(delta)) = 0;
 Vc = V .* exp(1j*delta);
 % Safety check:
for i = 2:no_bus
 if isinf(P(i))
 error(['Bus %d: P was entered as inf, but only the slack bus ' ...
'(bus 1) may have an unknown P. Every PV/PQ bus needs a ' ...
 'real P value -- re-run and enter the actual number.'], i);
 end
 end

isPV = isinf(Q);
```

```
 %% YBus formation
for i = 1:no_bus
 for k = i+1:no_bus
fprintf(sprintf('--- Line %d-%d (type inf,inf if no line) ---\n', i, k));
 r = input(sprintf('R(%d,%d) in pu: ', i, k));
 x = input(sprintf('X(%d,%d) in pu: ', i, k));
 y = 1/(r + 1j*x);
 Y(i,k) = Y(i,k) - y; Y(k,i) = Y(i,k);
 Y(i,i) = Y(i,i) + y; Y(k,k) = Y(k,k) + y;
 end
 end

 % Safety check:1 line connect
for i = 1:no_bus
 if Y(i,i) == 0
 error(['Bus %d has no connecting line (Y(%d,%d)=0), so its ' ...
 'voltage equation divides by zero. Check the R/X you ' ...
 'entered for every line touching this bus.'], i, i, i);
 end
end

fprintf('\n==================== BUS ADMITTANCE MATRIX (YBus) in pu
====================\n');
fprintf('%8s', '');
for k = 1:no_bus
 fprintf('%22s', sprintf('Bus %d', k));
```

```
 end
fprintf('\n');
for i = 1:no_bus
 fprintf('  Bus %d ', i);
 for k = 1:no_bus
 fprintf('   %9.4f %+9.4fj', real(Y(i,k)), imag(Y(i,k)));
 end
 fprintf('\n');
 end
fprintf('================================================================================\n');
 Ybus = Y

 %% iteration counter
iter = 0;
del_hist = [];
delta_hist = [];

 %% Gauss-Seidel sweep
del = 1;
while del >= 0.000170
 iter = iter + 1;
 del = 0;
 for i = 2:no_bus
 sum_term = Y(i,:)*Vc.' - Y(i,i)*Vc(i);
```

```
if isPV(i)
Q_new = -imag( conj(Vc(i)) * (Y(i,:)*Vc.') );
V_new = ( (P(i) - 1j*Q_new)/conj(Vc(i)) - sum_term ) / Y(i,i);
delta_new = angle(V_new);
del = max([del, abs(delta_new - delta(i))]);
delta(i) = delta_new;
Q(i) = Q_new;
else % (PQ bus)
V_new = ( (P(i) - 1j*Q(i))/conj(Vc(i)) - sum_term ) / Y(i,i);
V_mag_new = abs(V_new);
delta_new = angle(V_new);
del = max([del, abs(V_mag_new - V(i)), abs(delta_new - delta(i))]);
V(i) = V_mag_new;
delta(i) = delta_new;
end
Vc(i) = V(i) * exp(1j*delta(i));
if ~isfinite(V(i)) || ~isfinite(delta(i))
error(['Divergence at bus %d, iteration %d (value became ' ...
'NaN/Inf). Double check the R/X and P/Q/V/delta you ' ...
'entered -- a stray inf in the wrong field is the ' ...
'usual cause.'], i, iter);
end
end
% iteration's result save
del_hist(iter) = del;
delta_hist(iter,:) = delta*180/pi;
```

```
 end

 %% slack bus eqn
S1 = Vc(1) * conj( Y(1,:)*Vc.' );
P(1) = real(S1);
Q(1) = imag(S1);
fid = fopen('load_flow_result.txt', 'w');
fprintf(fid, 'GAUSS-SEIDEL LOAD FLOW RESULT\n');
fprintf(fid, 'Result found at iteration number: %d\n\n', iter);

fprintf(fid, 'BUS ADMITTANCE MATRIX (YBus) in pu\n');
for i = 1:no_bus
 for k = 1:no_bus
 fprintf(fid, '%9.4f %+9.4fj   ', real(Y(i,k)), imag(Y(i,k)));
 end
 fprintf(fid, '\n');
 end
fprintf(fid, '\n');

 fprintf(fid, 'Bus V(pu) Angle(deg) P(pu) Q(pu)\n');
for i = 1:no_bus
 fprintf(fid, '%3d %8.4f %10.4f %9.4f %9.4f\n', ...
 i, V(i), delta(i)*180/pi, P(i), Q(i));
 end

 fprintf(fid, '\nLINE FLOWS S(i,j) = P(i,j) + jQ(i,j) (pu)\n');
```

```
for i = 1:no_bus
 for k = i+1:no_bus
 if Y(i,k) ~= 0
 y_ik = -Y(i,k);
 Sik = Vc(i) * conj( (Vc(i)-Vc(k)) * y_ik );
 Ski = Vc(k) * conj( (Vc(k)-Vc(i)) * y_ik );
 fprintf(fid, '%d -> %d : %9.4f + j%9.4f\n', i, k, real(Sik), imag(Sik));
 fprintf(fid, '%d -> %d : %9.4f + j%9.4f\n', k, i, real(Ski), imag(Ski));
 end
 end
 end

fprintf(fid, '\nITERATION HISTORY (angles in degree)\n');
fprintf(fid, 'Iter');
for i = 2:no_bus
 fprintf(fid, '   delta%d(deg)', i);
 end
fprintf(fid, '   max change (del)\n');
for n = 1:iter
 fprintf(fid, '%3d ', n);
 for i = 2:no_bus
 fprintf(fid, '%14.4f', delta_hist(n,i));
 end
 fprintf(fid, '%18.6f\n', del_hist(n));
 end
fclose(fid);
```

```
fprintf('\nConverged at iteration %d. Results saved to load_flow_result.txt\n', iter);

%% Convergence graph
figure;
semilogy(1:iter, del_hist, '-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');
hold on;
plot([1 iter], [0.000170 0.000170], 'r--', 'LineWidth', 1.2);
text(1, 0.000170, '  Tolerance = 0.00017', 'Color', 'r', 'VerticalAlignment', 'bottom');
hold off;
grid on;
xlabel('Iteration number');
ylabel('Maximum change, del');
legend('del (largest change)', 'Tolerance', 'Location', 'northeast');
title('Convergence of Gauss-Seidel Load Flow');
set(gca, 'XTick', 1:iter);

%%  Bus voltage angles vs iterations
figure;
plot(1:iter, delta_hist(:,2:no_bus), '-o', 'LineWidth', 1.5);
grid on;
xlabel('Iteration number');
ylabel('Angle (degree)');
title('Bus voltage angles through the iterations');
legend(arrayfun(@(k) sprintf('\\delta%d', k), 2:no_bus, 'UniformOutput', false), 'Location',
'best');
```

```
set(gca, 'XTick', 1:iter);
```